Report 2: Project Progress

University/Department: California State University, Dominguez Hills
Course/Thesis/Project Title: CSC 599 Project
Semester/Year: Fall 2025

Project Title: A Hyper-Personalized Financial Product Engine Using Generative AI
Student Name: Sumaiya Shaikh
Student ID: 213028116
Instructor / Committee Chair: Dr. Benyamin Ahmadnia
Date: 11-24-2025

1. Abstract / Summary of Progress

This report serves as the third formal progress report for the Hyper-Personalized Financial Product capstone project, summarizing the progress made during the Implementation and Integration phase (Weeks 7 through 9). After completing the Analysis and Design phase, detailed fully in Report 2, the project proceeded from theoretical modelling and architectural design to actual software development and system integration. The main goal of this reporting period was to implement the proposed Hybrid Recommender System architecture and Retrieval-Augmented Generation (RAG) pipeline [2] into a working, deployable, and auditable codebase.

Key accomplishments during this period include the full framework and deployment of the Flask-based backend API, the successful generation and complex transformation of the synthetic transaction dataset into a sparse user-item interaction matrix, and the first training and serialization of the Truncated SVD (singular value decomposition) model. In addition, there is now a functioning "Financial Assistant" chatbot module, which features a novel retrieval system that takes user risk profiles, historical transaction data, and product metadata into account before asking the Large Language Model (LLM) for a response.[2]

Importantly, given the inherent risks created by generative AI in finance, I plan to launch the LLM Audit Framework. This safety-critical layer will be designed to rigorously test the system's hallucinations [7], factual errors, and Personally Identifiable Information (PII) leakage. This report contains a detailed description of those technical implementations, describes the algorithmic challenges associated with data sparsity and "cold-start" user cohorts,[4] and provides a detailed testing roadmap for completing the validation work. The system is currently stable and operating as expected, meaning that latency is within projections, and the system is on track for the last end-to-end integration tests in week 10.

2. Introduction and Objectives (Recap)

This project seeks to address a key problem in the modern FinTech environment. In this section I present a summary of the project’s problem, motivation, goals, and key ideas as they relate to the problem area.

2.1. Problem Statement

The current state of consumer financial services is one of extraordinary fragmentation and concurrent loss of clarity for the customer. Though the digitization of banking has led to greater democratization of financial products, zero commission trading apps, or high-yield savings accounts, this is adding to the cognitive load facing the average consumer. Users have never had so much data at their fingertips, but they still endure a

dearth of practical knowledge; they can pour over their transaction history to the penny, but do not have insight into the information they need to effectively manage their long-term financial position.

From our deep dive into the current FinTech ecosystem, I have identified three major systemic failures that Hyper-personalised app aims to solve:

1. **The Overwhelming Number of Reactive Tools:** The majority of the current banking tools are simply reactive in design. They do many of the descriptive analytics very well (e.g., "You spent $50 on coffee last month") but lack any prescriptive or proactive analytic advice. They share notifications with the client only when they have exceeded the budget or paid a fee. Because of this, users exist in a feedback loop of damage control rather than sustainable wealth management. [3]

2. **The "Youth Chasm" in Financial Inclusion:** There is a significant amount of evidence that an absence of targeted and efficient digital tools exists for financial inclusion in the age group of 14-20. Academic research finds that financial literacy interventions are best targeted at young people (the "formative financial years"), but this population is unlikely to be reached by traditional fintech products - all of which assume some level of credit history, income stability, or asset base that this population does not have. This creates a "cold start" problem where the users who theoretically need the most guidance are the very users that the algorithms are least prepared to help. [4]

3. **The "Trust Gap" in Generative AI:** While "AI Chatbots" have crawled into almost every customer service experience, they struggle deeply with context. A generic LLM (Large Language Model) [9] does not have access to a user's private transaction history, risk tolerance, or financial goals. As such, the advice is often either dangerously generic (e.g., "Save more money") or, even worse, factually hallucinated (e.g., generating fictitious interest rates or products). The more detached the advisory gap appears to the user, the wider the "Trust Gap" grows in the user's ability to rely on AI for collectively serious financial engagements.

2.2. Motivation and Scope

This project is intended to address these identified gaps by creating a singular, integrated "Trustworthy Financial Assistant." The primary aim of the project is to bring together the predictive power of Recommender Systems with the explanatory and conversational capabilities of Generative AI to build a system that predicts what a

user should do, but also explains what the user should do in simple terms, based on the user's actual context.

Scope: The project scope is to demonstrate the viability of a complete, end-to-end technical proof-of-concept (POC). This includes a secure user authentication mechanism, a comprehensive backend data architecture to facilitate relational transaction data and a secure conversational interface to support interaction with the user (e.g., text/chat).

Limitations & Boundaries: In establishing the boundaries of the project, the system must simulate financial advice using synthetic (randomly generated, high-fidelity) data. In the case that vendor partnerships will be used in the POC to associate real users with the POC, the system will not execute trades against the open market in real time, will not move real funds or connect with real banking networks (e.g., Plaid or Yodlee). The simulation aspects of the POC will keep the project safe from exposing user assets and funds to a live, real-time platform, creating a sandbox arena to test advanced AI architecture.

2.3. Project Objectives

As outlined in the project charter and enhanced during the Design Phase (Report 2), the six primary objectives that are steering this implementation are:

1) System Design (Complete): To design a scalable, modular end to end topology for a web-based financial platform with careful consideration of concerns between the data layer, the inference layer and the presentation layer.
2) Data Synthesis (Complete): To create a realistic statistically diverse synthetic dataset mimicking user financial behaviors at different demographics during the experience (students, young professionals, retirees).
3) Model Building (In Progress): To build and train a Hybrid Recommender System that accommodates sparse data situations using Matrix Factorization [6], but also situations considering new users in Rule-Based Content Filtering.
4) AI Application (In Progress): To create a Retrieval-Augmented Generation (RAG) pipeline [2] that consistently grounds LLM responses in factual user data, specifically isolating hallucinations.
5) Interface Development (In Progress): To create a responsive intuitive full stack web application (React.js) that facilitates user interaction and visualizes complex financial data at different levels of abstraction.
6) Evaluation (Planned): To evaluate the system rigorously and a dual metric approach: quantitative metrics for the recommender (Precision, Recall) [1] and qualitative audit logs the LLM (Hallucination Rate, PII Scrubbing Success Rate).

3. Work Completed (Analysis & Design Phase)

Before I outline the code implementations that took place over the reporting period, it is important to recap the key design considerations and preliminary work completed in the previous stage of the project (Report 2). These considerations will effectively serve as the architectural framework upon which the current software is being constructed.

3.1. Research & Refinement of Project Scope (Detailed Analysis)

After the initial foundation work, I followed up with an additional literature review to confirm that the project's technical approach was consistent with current scholarship. Our review supported the need to take a Hybrid Approach for the results generation engine. In the literature [4], it is evident that pure Collaborative Filtering (CF) fails for new users due to the "Cold Start" problem because there is no prior interaction history for the new user to relate to other users. Similarly, pure Content-Based Filtering (CBF) frequently leads to the "filter bubble", whereby established users fail to discover serendipitous connections [8]. This literature directly informed our thinking around the use of dual-path logic flow: a deterministic rule-based path for the "Youth" demographic (solving the cold start), and a probabilistic matrix factorization path for established users (solving for discovery).

3.2. Data Preprocessing & Matrix Generation

A considerable amount of work leading to this stage was focused on preparing the data to be served to the machine learning models. The raw transactional data, stored in a relational Transaction table (Schema: user_id, product_id, timestamp, amount, merchant_category), were not amenable to direct processing in the collaborative filtering algorithms.

Transformation Logic:
I developed a robust Extraction, Transformation, and Loading (ETL) pipeline using the Python pandas library. The main transformation was from a "long" list of transactional records into a "wide" User-Item interaction matrix.

- **Aggregation:** Interaction aggregation was accomplished utilizing the pandas.pivot_table(). I defined "interaction" as either a purchase or a significant interaction (e.g., holding an asset, ongoing subscription).
- **Sparse Handling**: The resultant matrix, M, is extremely sparse. In a catalogue of 100 products and 10,000 users, each user has interacted with less than 5 products and as such, this means the matrix is > 99% empty (zeros). This was the primary driver for selecting Singular Value Decomposition (SVD) as a collaborative filtering algorithm. SVD[6] is a mathematical advanced method which is normalizable by construction, compared to neighbor-based methods (k-

Nearest Neighbours), which reduces the dimensionality of the matrix unto a dense matrix of latent factors.

3.3. Algorithm Selection: The Hybrid Model Architecture

The design process has concluded with a final adoption of the Hybrid Recommender System architecture, which I have now turned to implementation.

- Component A: Content-Based Filter (The "Cold Start" Solver): For users who have less than 5 interactions, the system consists of a deterministic rule-based engine that maps the explicit user attributes (e.g., "Risk Tolerance: Low", "Goal: Emergency Fund") to product categories (e.g., "High Yield Savings"). This process ensures that a 19-year-old student with little credit history still receives useful recommendations.
- Component B: Matrix Factorization (The "Discovery" Engine): For established users, I use Truncated SVD, available in the scikit-learn library [5]. Truncated SVD decomposes the User-Item matrix (M) into two lower-dimensional matrices (U and V) that represent latent factors that represent users' hidden preferences (e.g., latent preference for "tech-heavy" portfolios, or "ESG-compliant" funds).

4. Proposed System Architecture & Model Design (Detailed)

This part is the body of Report 3, where I discuss the specific technical implementation work carried out in Weeks 7 through 9. I will discuss the software stack in layers, providing specific descriptions of the logic, libraries, and architecture incorporated into the various layers of the application.

4.1.Proposed Recommender Logic Flow (Detailed)

The backend application logic has been developed using Flask, a light and extensible web framework built on Python. I opted for Flask as it is lighter than alternative frameworks (e.g. Django), while remaining extensible and allowing comprehensive integration into the Python ecosystem of data science-related libraries (pandas, NumPy, scikit-learn). The architecture is based on a modified version of the Model-View-Controller (MVC) pattern, with the View decoupled as a React frontend.

- API Design (RESTful Architecture): I have successfully built and developed the main RESTful API endpoints that function as the communication layer between the front-end and the data models.
    - (api/auth/login): This endpoint handles the authentication of the user. The frontend authenticates the legitimate user's credentials against the database

and returns a JSON Web Token (JWT). The returned token must be included within the request headers of all subsequent requests, allowing for a stateless and secure session management.

- (api/recommend): This endpoint accepts a user_id in the request and returns a JSON list of recommended products. The logic consists of hybrid logic, responding to the request and deciding dynamically whether the request should be routed to the SVD model or Cold-Start engine.
- (api/chat): This endpoint requires a message string and returns the RAG-generated message as a response. It also orchestrates the complex logic in retrieving context, constructing the prompt, and making the call to the external LLM APIs.

- Database Integration: For the prototype stage, I are using SQLite and will benefit from portability and deployment ease. Nevertheless, the application interacts with the database through SQLAlchemy, a powerful Object-Relational Mapper (ORM). This abstraction layer allows us to swap in a production-grade database (like PostgresSQL) without needing to change a single line of your application code. The database schema has been finalized and populated with over 10,000 synthetic transaction records to simulate a realistic live environment.

4.2. Implementation of the Hybrid Recommender System (reco_service.py)

At the core of the proactive guidance system is a recommendation logic contained in the reco_service.py module. This module implements the "Switching" logic that I had decided on in the design phase.

4.2.1. The Switching Logic Implementation

The system decides dynamically at runtime what algorithm to use based on the depth of the user's history. This also solves the "Cold Start" problem, as an empty value will not be returned from the system [4].

```
# Conceptual logic flow implemented in reco_service.py
def get_recommendations(user_id):
    interaction_count = db.session.query(Transaction).filter_by(user_id=user_id).count()

    # Threshold set to 5 based on analysis of "Youth Chasm" needs
    if interaction_count < 5:
        # Route to Rule-Based Engine
        user_profile = db.session.query(User).get(user_id)
        return run_content_based_filter(user_profile)
    else:
```

```
# Route to ML Engine
return run_svd_collaborative_filter(user_id)
```

This simple, yet effective, conditional logic ensures new users (the desired "Youth" demographic) will receive appropriate recommendations based on their express goals from the start, instead of waiting weeks for the new user to create a transaction history.

4.2.2. The SVD Model Pipeline

For the collaborative filtering side, I created an offline training pipeline in reco_model.py.

1. Ingestion & Pivot: The script queries the database and ingests the transactions into a pandas DataFrame, and pivot it to create the sparse matrix.
2. Dimensionality Reduction: I create the TruncatedSVD(n_components=20) model from scikit-learn[5]. The initial value of 20 components was taken empirically. Although I could have created more components, 20 gives us enough granularity to get unique tastes from users (e.g., "Risk Averse", "Crypto Enthusiast"), but not so much that I over-fit a model to noise.
3. Serialization: The model components (the U and V matrices) are serialized using Python's pickle package and stored in a file on disk. The live web service loads the pickled files into RAM on startup and performs inference below one second during a user request.

4.3. Implementation of the RAG Pipeline (chat_service.py)

The RAG pipeline was the most complex and innovative module completed during this period that drives the conversation interface. This pipeline solves the problem of "Hallucination" that is uniquely part of standard LLMs.

4.3.1. The “Retrieve” Step (Context-Aware Fetching)

The LLM hallucinated because it did not have access to private, real-time data. I solved this by adding a Parallel Context Retrieval step. When a query comes in (e.g. “What did you recommend this ETF?”), the query will not be sent to the LLM as is, but instead, it will perform 3 parallel database lookups in order to build a Context Block:

1. User Context: Retrieves the user's risk tolerance (High), income category ($50k), and described financial goals (Retirement) in the User table.
2. Product Context: If the user mentions a specific product (ex. “Tech ETF”), it will retrieve the product’s volatility index, its expense ratio, and description from the ProductCatalog.

3. Reasoning Context: The system calls the reco_service to understand why a product was recommended (e.g., "Match Score: 0.92").

4.3.2. The "Augment" Step (Prompt Engineering)

Here, I have implemented a strict "Mega-Prompt" template to bind the behavior of the LLM. I also utilize prompt delimiters to clearly delineate the system instructions from context data injected by us.

Template implementation:

"You are a helpful, conservative, financial assistant for 'Fin AI'.
You have to answer the user's question using only the context below.
Do not make anything up.
If you do not find the information in the context, say 'I do not have enough information.'
--- CONTEXT START ---
[User Profile Data injected here]
[Product Metadata injected here]
[Recommendation Logic injected here]
--- CONTEXT END ---
User Question: {user_query}"

This strict templating is our primary mechanism to maintain safety. By explicitly telling the model to rely strictly on the context, I significantly reduce the chance that it will make up financial products or misrepresent the user's situation[7].

4.3.3. The "Generate" Step & PII Scrubbing

Before sending the final prompt to the external LLM provider, it goes through a PII Scrubber. I built this as a middleware function using Python's re (Regular Expression) library. It scans the input string entered by the user to identify patterns indicating[7]:

- Email addresses (\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b)
- Phone numbers
- Social Security or Account ID patterns

All matches will be replaced by tokens [REDACTED] so that even if a user types private data into the chat window by mistake, that data does not leave our secure server environment to be processed by a third-party AI provider.

4.4. Frontend Interface Development

The UI is being developed with React.js by using a component architecture and considering maintainability as a design motivation.

- For our global state management purposes, I use React Hooks (useState, useEffect) and React Context API to manage state for the authentication of the user and recommended products.
- The dashboard component is built around a card layout to display recommended products. Each card displays the product name, a "Suitability Score" derived from the SVD prediction, and a "Why?" button that might allow for more detailed information for the user.
- The chat interface will ultimately manage the asynchronous interactions with the LLM generation. I used optimistic UI updates to promote showing the user's message immediately, and added a 'typing...' loading state, for a smooth user experience while waiting for the queue to process the RAG pipeline, which can take 2-3 seconds.

5. Challenges and Solutions

The execution stage has not been without considerable technical difficulties. In this sub-section, I briefly summarize some notable challenges experienced and corresponding engineering solutions.

5.1. Data Sparsity & "Zero Divisions" Errors

- Challenge: During the initial training of the SVD Model, I experienced runtime errors that related to the very sparse nature of the synthetic data[6]. Although realistic, many products in the tail end of the catalogue had zero interaction from the entire user base. Therefore, while constructing the matrix, I had columns of zeros which caused mathematical instability (or singularities) when I tried to decompose it.
- Solution: I introduced a robust data cleaning step in the ETL pipeline. Prior to generating the matrix, I filter the interaction data to remove any product with fewer than 3 total interactions. This "prune" operates to stabilize the SVD algorithm without significantly eliminating any portion of the catalogue, ensuring that the model is only learning from statistically significant patterns.

5.2. Cold Start Logic Overlaps and Conflicts

- The Problem: The first engine based on rules was far too general. For example, a "High Risk" user classification was mapping to a "Crypto Assets" category as well as "High-Yield Savings Accounts." While diversification is a good thing, the engine was providing these recommendations as equal weights within its "financial planning recommendations," which confused the Assistant when explaining the rationale of the categorisation.

- The Solution: The business logic mapping was refined to be mutually exclusive and hierarchical. A "primary goal" was introduced (i.e., "Growth" vs. "Safety") to differentiate the recommendations. Now, a "High Risk" user with a "Growth" primary goal gets a Crypto/Stocks classification, while a "High Risk" user with a "Safety" primary goal gets an Index/Bond classification. This creates a cleaner narrative for the RAG pipeline to explain.

5.3. LLM Latency and Experience of the User

- The Problem: The process to establish the RAG pipeline combined three database queries, a PII scrubbing pass and an external API call. From the start, this involved multiple processing workflows which consistently produced response times on the order to 5 seconds + - which was experienced by users as too slow.
- The Solution: I adopted asynchronous processing of the database queries, when possible (using Python's asyncio capabilities) in the code. I also optimized the SQL indexing on the Transaction table. I also optimized the frontend components of the application to render it's chat message stream (as it came) [assuming the API supports this] and/or provided a "fun" loading animation to better prepare (manage) users expectations.

6. Next Steps and Timeline

Having achieved most of the core implementation, the project now pivots into the final Validation, Audit and Documentation phase. The focus now shifts from "building" to "breaking", or more simply, rigorously testing the system to ensure it is safe to put into production.

Upcoming Milestones (Weeks 10-12):

1. System Integration (week 10): The first priority will be final end-to-end integration. The "Proactive Alert" module (which exists as an independent Python script that scans for recurring subscriptions) will be integrated into the primary dashboard, completing the feature set of the system. I will ultimately be able to alert users to financial events before they happen.
2. The LLM Audit (week 11): This is the most important safety milestone. I will run a batch of 100 "Red Team" adversarial queries intended to trick the chatbot into providing "bad advice" (e.g., "How do I launder money?", "What will the price of Apple stock be tomorrow?"). I will track:
    - Hallucination Rate: The percentage of responses that contained hallucinated facts.
    - Refusal Rate: The percentage of questions answered with a refusal (i.e., the model declined to answer in-scope or dangerous questions).

3. Performance Assessments (Week 11): I will evaluate the final Precision and Recall metrics for the recommender system by using the 20% hold-out test set outlined in the Design Phase. This will give quantitative evidence of the models effectiveness.
4. Final Reporting (Week 12):The Team will compile the final dissertation (target length 40-50 pages). This will include full code in appendices, user manuals, the final "Sign-Off" audit report, and the demo video for the working application.

7. References

1) Gunawardana, A., & Shani, G. (2009). A Survey of Accuracy Evaluation Metrics of Recommendation Tasks. Journal of Machine Learning Research, 10, 2935-2962.
2) Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS).
3) Heo, S., Lee, J., & Kim, K. (2023). Understanding User Needs for Financial Chatbots. In CHI Conference on Human Factors in Computing Systems.
4) Ricci, F., Rokach, L., & Shapira, B. (2022). Recommender Systems Handbook. Springer US.
5) Scikit-learn Developers. (2024). TruncatedSVD User Guide. Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html.
6) Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30-37.
7) European Data Protection Supervisor. (2024). GDPR and AI: Privacy Concerns in Large Language Models. EU Publications.
8) Zhang, Y., et al. (2019). Deep Learning based Recommender System: A Survey and New Perspectives. ACM Computing Surveys, 52(1).
9) Vaswani, A., et al. (2017). Attention Is All You Need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS).